Bayesian Movie Recommendation with Probabilistic Matrix Factorization

Bashar Bdewi – s183356
42186 Model-Based Machine Learning
Technical University of Denmark

***Abstract*—Movie recommendation is a natural model-based machine learning problem because the observed data only contains a small subset of all possible user–movie ratings. This project studies a Bayesian matrix factorization model for predicting MovieLens ratings. The model assumes that each user has a latent preference vector and each movie has a latent attribute vector. A rating is then generated from a global mean, user and movie bias terms, and the inner product between the corresponding latent vectors. The model is implemented in Pyro and fitted using stochastic variational inference with an AutoNormal guide. Two simple baselines are used for comparison: a global mean predictor and a regularized user/movie bias model. On a MovieLens subset with 250 users, 400 movies, and 29,368 observed ratings, Bayesian matrix factorization achieves the best predictive performance with RMSE 0.817 and MAE 0.619. The improvement over the bias baseline is modest but consistent, and the posterior predictive checks show that the model captures the overall rating distribution well. The project therefore demonstrates how a simple probabilistic graphical model can give both competitive predictions and uncertainty estimates for recommender systems.**

I. Introduction

Recommender systems try to predict what a user may like based on previous interactions. In the movie rating setting, the data can be seen as a large user–movie matrix where most entries are missing. A user usually rates only a small number of movies, and a movie is usually rated only by a subset of users. This makes the problem suitable for matrix factorization: instead of learning one independent parameter for every possible user–movie pair, we learn a compact representation of users and movies in a lower-dimensional latent space. Matrix factorization is a classical approach for recommender systems [1].

The goal of this project is to build a small but complete model-based machine learning solution for movie rating prediction. The research question is whether a Bayesian matrix factorization model can predict ratings better than simple baseline models while also providing useful uncertainty information. This follows the model-based machine learning workflow of defining a generative process, performing inference, and checking the model empirically [2].

The project uses the MovieLens dataset, which is a standard benchmark for recommender systems [3]. A Bayesian formulation is used instead of a purely deterministic factorization model. The main reason is that probabilistic modelling is not only about point predictions, but also about assumptions, uncertainty, and the generative process behind the observed data [4]. In this project, the assumptions are kept deliberately simple so that the model remains interpretable and feasible to run on a laptop.

TABLE I
Project setup after preprocessing.

Item	Value
Users	250
Movies	400
Observed ratings	29,368
Train ratings	23,494
Test ratings	5,874
Latent dimension	3
Observed density	29.4%

II. Data and Preprocessing

A. MovieLens subset

The raw data consists of user IDs, movie IDs, ratings, and timestamps. The rating values are on a half-star scale from 0.5 to 5.0. Since the full MovieLens matrix is sparse, the project uses a denser subset built from the most active users and the most rated movies. This makes training faster and reduces the number of cold-start cases where a user or movie has very little information.

The final subset contains 250 users, 400 movies, and 29,368 observed ratings. The user–movie matrix has 100,000 possible entries, so the observed density is about 29.4% and the sparsity is about 70.6%. The observed ratings were split into 23,494 training ratings and 5,874 test ratings. The split was made over observed rating rows, not over users or movies, so the test set measures how well the model predicts held-out ratings for users and movies that appear in the training set.

Figure 1 shows the main exploratory plots. The rating distribution is concentrated around 3, 4, and 5 stars, which means the data is not centered around a neutral value. The counts per user and per movie are highly uneven. A few users and movies have many ratings, while many others have much fewer. This long-tail behavior is a typical recommender-system pattern and motivates the use of bias terms and latent factors.

The uneven amount of information per user and per movie is also important. A model can be more confident about users and movies that appear many times in the training set, but it has less evidence for rare users and movies. To make this project manageable, the subset was selected from active users

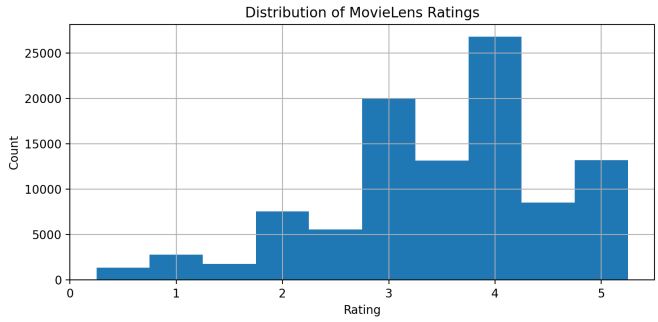

Fig. 1. Distribution of MovieLens ratings in the selected dataset. Ratings are mostly positive, with a high concentration around 3 to 5 stars.

and popular movies. This does not remove sparsity completely, but it gives enough repeated observations for each user and movie to learn meaningful bias terms and latent factors. Figure 2 shows this long-tail pattern. Even after subsetting, many movies and users have relatively few ratings, while a smaller number have many.

B. Train-test representation

For Pyro and PyTorch, the original user and movie IDs were mapped to consecutive integer indices. Each observed row is represented as a triple (i_n, j_n, r_n), where i_n is the encoded user index, j_n is the encoded movie index, and r_n is the observed rating. This representation is convenient because latent factors can be stored as matrices and indexed directly during mini-batch training.

III. Models

A. Baseline models

Two baselines are used before fitting the Bayesian model. The first baseline predicts every test rating using the global mean of the training ratings. This is intentionally very simple, but it is useful because any reasonable recommender model should improve over it.

The second baseline is a regularized user/movie bias model. It predicts a rating using

$$\hat{r}_{ij} = \mu + b_i^{(u)} + b_j^{(m)}, \tag{1}$$

where μ is the global mean, $b_i^{(u)}$ is a user bias, and $b_j^{(m)}$ is a movie bias. The user bias captures whether a user tends to give high or low ratings in general. The movie bias captures whether a movie tends to be liked or disliked in general. These parameters are estimated by alternating regularized updates. This baseline is stronger than the global mean because it uses both user-specific and movie-specific average effects.

B. Bayesian matrix factorization

The Bayesian model extends the bias baseline by adding latent user and movie vectors. The intuition is that bias terms explain simple average behavior, while latent vectors explain hidden interactions. For example, a user may like science fiction movies more than romantic comedies, but this preference is not directly observed as a variable in the dataset. Matrix factorization tries to represent such hidden structure in a low-dimensional latent space.

For each user i, the model contains a user bias $b_i^{(u)}$ and a latent vector $u_i \in \mathbb{R}^K$. For each movie j, it contains a movie bias $b_j^{(m)}$ and a latent vector $v_j \in \mathbb{R}^K$. For an observed rating row n, the expected rating is

$$\eta_n = \mu + b_{i_n}^{(u)} + b_{j_n}^{(m)} + u_{i_n}^\top v_{j_n}. \tag{2}$$

The dot product $u_{i_n}^\top v_{j_n}$ is the hidden interaction between the user and the movie.

The full generative process is

$$\mu \sim \mathcal{N}(3.5, 1^2), \tag{3}$$

$$\sigma \sim \text{HalfCauchy}(1), \tag{4}$$

$$b_i^{(u)} \sim \mathcal{N}(0,1), \quad b_j^{(m)} \sim \mathcal{N}(0,1), \tag{5}$$

$$u_i \sim \mathcal{N}(0, I_K), \quad v_j \sim \mathcal{N}(0, I_K), \tag{6}$$

$$r_n \sim \mathcal{N}\left(\mu + b_{i_n}^{(u)} + b_{j_n}^{(m)} + u_{i_n}^\top v_{j_n}, \sigma^2\right). \tag{7}$$

The latent dimension is set to $K = 3$ in the final experiment. This is small enough to train quickly while still allowing more structure than the bias baseline.

The Gaussian likelihood is a modelling approximation because MovieLens ratings are discrete half-star values. It is still a reasonable first model because ratings are ordered and approximately continuous. The predictions are clipped to the valid rating range $[0.5, 5.0]$ during evaluation.

The priors are intentionally weak but regularizing. The prior on μ is centered at 3.5 because MovieLens ratings are usually positive and the training mean is close to this region. The standard normal priors on biases and latent factors prevent the model from assigning arbitrarily large values to individual users or movies. This is important because matrix factorization can otherwise overfit sparse entries. The half-Cauchy prior on σ keeps the observation noise positive while allowing a range of possible noise levels.

There is also a mild identifiability issue in matrix factorization: rotating the latent user and movie vectors can leave the dot products almost unchanged. This is not a major problem for prediction, because the dot product is what matters. It does mean that individual latent dimensions should not be interpreted too literally. The latent space plots in the notebook are therefore used as qualitative diagnostics, not as formal evidence that a specific dimension corresponds to a human-interpretable genre.

IV. Inference and Implementation

The posterior distribution over all parameters is not available in closed form. This includes the global parameters, all user and movie biases, and the latent factor matrices. The model is therefore trained using stochastic variational inference (SVI). The implementation uses Pyro, a probabilistic programming framework built on PyTorch [5]. The variational approximation is an AutoNormal guide, which gives a mean-field Gaussian approximation to the posterior.

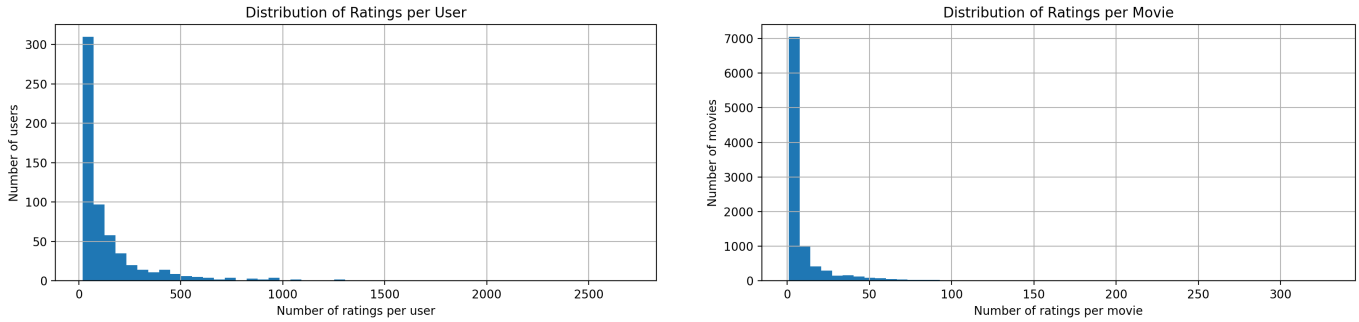

Fig. 2. Number of ratings per user and per movie. The distributions are long-tailed, so a few users and movies contain much more information than the rest.

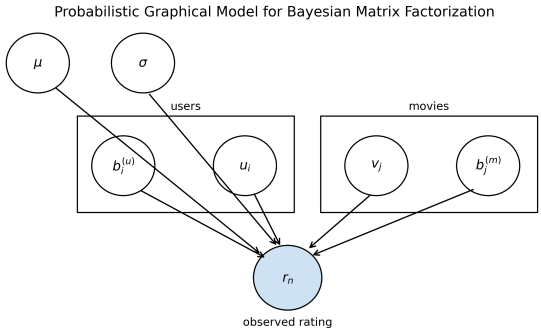

Fig. 3. Probabilistic graphical model for Bayesian matrix factorization. The observed rating depends on global parameters, user-specific variables, movie-specific variables, and the latent user–movie interaction.

The objective optimized by SVI is the evidence lower bound (ELBO):

$$\mathcal{L}(q) = \mathbb{E}_{q(\theta)}[\log p(r, \theta) - \log q(\theta)], \tag{8}$$

where θ denotes all latent variables and parameters in the model. Maximizing this objective approximately fits the posterior. In the notebook, the reported training curve shows the normalized negative ELBO, so lower values are better.

Mini-batches of rating rows are used during training. This is important because each SVI update does not need to use all 23,494 training ratings. The batch size is 1024, the number of SVI steps is 1500, and the optimizer is ClippedAdam with learning rate 0.03. The settings are summarized in Table II. Figure 4 shows that the loss decreases sharply in the first part of training and then stabilizes around 1.3–1.4. This indicates that inference is learning a stable posterior approximation.

After training, posterior samples are drawn from the variational guide. For each test rating, the model produces multiple posterior predictive samples. The posterior predictive mean is used as the final prediction, while the posterior predictive standard deviation is used as a measure of uncertainty.

TABLE II
TRAINING AND INFERENCE SETUP.

Item	Value
Inference method	SVI
Guide	AutoNormal
Loss	Trace ELBO
Optimizer	ClippedAdam
Learning rate	0.03
Mini-batch size	1024
SVI steps	1500
Posterior samples	200

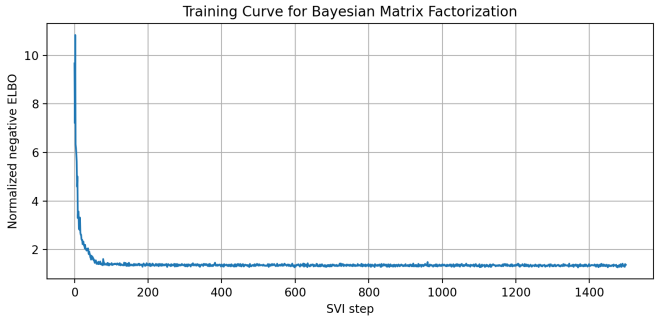

Fig. 4. Training curve for Bayesian matrix factorization. The normalized negative ELBO decreases quickly and then stabilizes, which suggests that SVI has found a reasonable variational solution.

V. EVALUATION

A. Predictive performance

The main evaluation metrics are root mean squared error (RMSE) and mean absolute error (MAE). Both are computed on the held-out test ratings. Lower values indicate better prediction. Table III shows the final comparison.

The global mean baseline performs worst. This is expected because it ignores both user behavior and movie popularity. The user/movie bias baseline improves the results substantially, which shows that much of the predictive signal is already explained by simple average effects. Bayesian matrix factorization gives the best performance, with RMSE 0.817 and MAE 0.619. The improvement over the bias baseline is not large, but it is consistent for both metrics. This suggests that the latent interaction term captures additional information

TABLE III
TEST-SET PERFORMANCE. LOWER RMSE AND MAE ARE BETTER.

Model	RMSE	MAE
Bayesian matrix factorization	**0.817**	**0.619**
User/movie bias baseline	0.831	0.632
Global mean	0.972	0.768

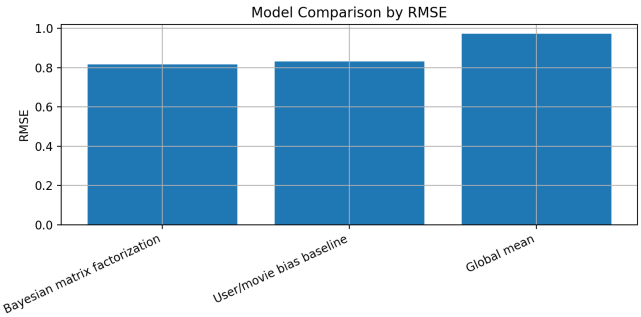

Fig. 5. RMSE comparison across the three models. Bayesian matrix factorization gives the lowest error, followed closely by the user/movie bias baseline.

beyond user and movie averages. Figure 5 visualizes the same result for RMSE.

Figure 6 shows predicted ratings against true ratings. The plot shows the typical regression-to-the-mean behavior of recommender models: extreme low and high ratings are harder to predict exactly, while predictions often concentrate around the middle and upper-middle rating range. This is also visible in the residual distribution in Fig. 7. The residuals are centered close to zero, which indicates no large global bias, but there are tails for difficult examples.

B. Posterior predictive check

A useful advantage of the Bayesian model is that it can generate posterior predictive ratings. This makes it possible to check whether the model can reproduce basic properties of the test data. Figure 8 compares the distribution of true test ratings with simulated posterior predictive ratings. The two distributions are not identical, but they are close in overall shape. The posterior predictive mean is 3.66, compared with a true test mean of 3.71. The posterior predictive standard deviation is 0.95, compared with a true test standard deviation of 0.97.

The uncertainty intervals in Fig. 9 also show an important property of the model. Some predictions have wide intervals, meaning the model is uncertain about the rating. This is useful in a recommendation setting because uncertainty can help distinguish confident recommendations from weak ones. At the same time, the intervals are quite wide for many examples, which suggests that the posterior approximation and the Gaussian likelihood may be conservative.

C. Latent factor inspection

The posterior mean latent factors can be plotted in their first two dimensions. These plots should be interpreted carefully

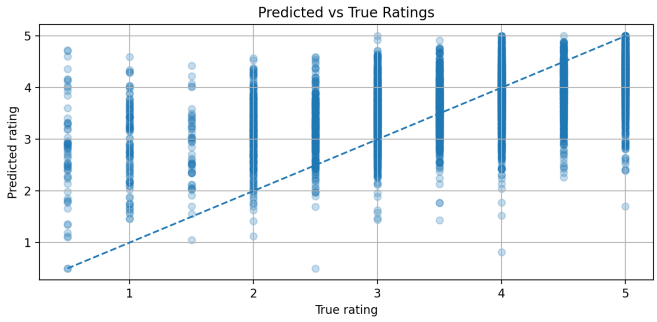

Fig. 6. Predicted versus true ratings for the Bayesian model. The dashed line is the ideal prediction line. The vertical bands appear because true ratings take discrete half-star values.

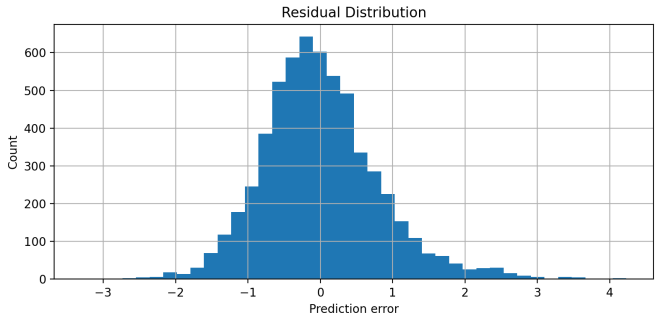

Fig. 7. Residual distribution for Bayesian matrix factorization, computed as predicted rating minus true rating. The distribution is centered close to zero, but some difficult cases remain.

because factor models are not uniquely identifiable up to rotations and sign changes. Still, the plots are useful as sanity checks. If all users or movies collapsed to one point, it would suggest that the latent factors were not being used. In Fig. 10, both users and movies spread across the latent space, which is consistent with the model learning some variation beyond simple bias terms.

VI. DISCUSSION

The results show a clear progression from simple to more expressive models. The global mean baseline is too simple because it treats all users and movies as identical. The bias baseline is much stronger because it separates general user behavior from general movie popularity. The Bayesian matrix factorization model adds latent interactions and achieves the best RMSE and MAE.

However, the improvement over the bias baseline is modest. There are several possible explanations. First, the selected subset is relatively dense and contains active users and popular movies, so user and movie averages are already strong predictors. Second, the latent dimension was kept small for simplicity and reproducibility. A larger latent dimension or more careful hyperparameter tuning might improve performance, but it would also increase the risk of overfitting and longer training time. Third, the likelihood treats ratings as Gaussian continuous values, although the observed ratings are discrete

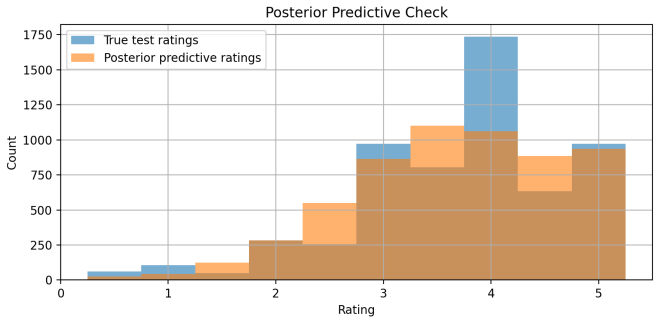

Fig. 8. Posterior predictive check. Simulated ratings from the fitted model are compared with the true test ratings. The overall mean and spread are close.

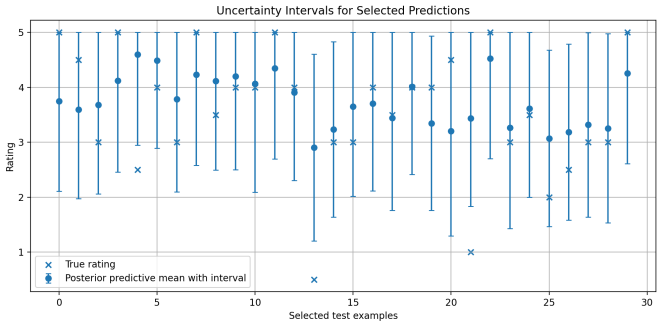

Fig. 9. Posterior predictive intervals for selected test examples. Crosses show true ratings and circles show posterior predictive means with approximate 95% intervals.

TABLE IV
POSTERIOR PREDICTIVE SUMMARY STATISTICS.

Statistic	True test	Predictive
Mean	3.711	3.664
Std. deviation	0.972	0.951
Minimum	0.500	0.500
Maximum	5.000	5.000

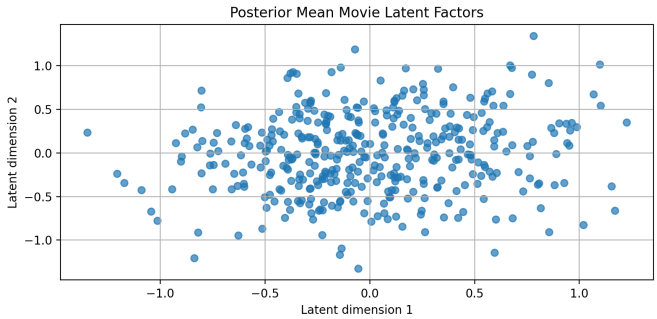

Fig. 10. Posterior mean movie latent factors in the first two latent dimensions. This is a qualitative diagnostic rather than a direct semantic interpretation.

half-star values. An ordinal likelihood could be a more faithful model.

The project also has some limitations. It does not handle cold-start users or movies, because all test users and movies are assumed to have appeared in training. It uses SVI and a mean-field AutoNormal guide, which may underestimate posterior dependencies between parameters. It also does not use movie genres, timestamps, or user metadata. These variables could be added in a richer hierarchical model, for example by letting movie latent factors depend partly on genre information or allowing user preferences to change over time.

From a model-based machine learning perspective, the strongest part of the project is that the assumptions are explicit. The PGM states exactly how ratings are assumed to be generated, and the posterior predictive check tests whether the fitted model can reproduce the observed rating distribution. This makes the model more interpretable than a black-box predictor, even if it is still relatively simple.

A useful practical lesson is that the baseline matters. Without the bias baseline, the Bayesian matrix factorization model would look much stronger because it improves clearly over the global mean. With the bias baseline included, the conclusion becomes more realistic: latent factors help, but a large part of the signal in this subset is already explained by average user and movie effects. This is a better scientific result because it shows what the Bayesian model actually adds.

Another important point is the difference between predictive accuracy and model checking. RMSE and MAE tell us whether the point predictions are close to the observed test ratings. The posterior predictive check asks a different question: whether data generated from the fitted model looks similar to the real data in distribution. In this project, both views are useful. The RMSE/MAE results show a small improvement in point prediction, while the posterior predictive check shows that the model captures the overall mean and spread of ratings reasonably well.

To keep the experiment reproducible, I used the same preprocessing and train-test split throughout the notebook and report. The subset size was not chosen to maximize performance, but to make the complete probabilistic workflow run reliably on a standard laptop. This was important for the project because the aim was not only to report a final error number, but also to show a transparent path from data to graphical model, inference, posterior prediction, and model checking. The notebook therefore saves the main figures and result tables directly from the executed code, and these are the same outputs used in this report.

The uncertainty estimates should also be understood in this practical setting. The posterior predictive intervals are useful because they show when the model is less certain, but they are not perfect calibrated recommendation intervals. A mean-field guide assumes independent variational factors, so it can miss correlations between user factors, movie factors, and the noise parameter. Still, the intervals are informative enough to show that the Bayesian version gives more than a single deterministic prediction. This is one of the main reasons for using a model-based approach instead of only fitting a deterministic matrix factorization model.

A final modelling choice concerns the selected latent di-

mension. I used a small value, $K = 3$, because it keeps the latent interaction term simple and makes the PGM easier to explain. Larger values could reduce error slightly, but they would also make the latent representation harder to inspect and could overfit the selected subset. For a course project completed individually, this trade-off was acceptable: the model is expressive enough to improve over the bias baseline, but still simple enough that every variable in the generative story has a clear role.

VII. Conclusion

This project implemented a Bayesian matrix factorization model for MovieLens rating prediction. The model combines a global mean, user and movie biases, and latent user–movie interactions in a probabilistic graphical model. It was fitted using Pyro and stochastic variational inference with an AutoNormal guide. Compared with two baselines, the Bayesian model achieved the best test performance, with RMSE 0.817 and MAE 0.619. The posterior predictive check showed that the fitted model produced ratings with a similar mean and spread to the real test ratings.

The final model is intentionally simple, but it demonstrates the main idea of model-based machine learning: define a generative story, encode it as a probabilistic model, perform inference, and evaluate both predictions and uncertainty.

VIII. Future Work

There are several natural extensions of this project. The first would be to replace the Gaussian likelihood with an ordinal likelihood that respects the half-star rating scale. This would make the observation model closer to the real data-generating process, because a rating of 4.0 is an ordered category rather than a continuous measurement. Another extension would be to use movie genres, tags, or release years as covariates. This could help the model share information across movies, especially for movies with fewer ratings.

A second direction would be to test richer Bayesian inference choices. The current AutoNormal guide is simple and fast, but it assumes a factorized Gaussian posterior. A more expressive guide could capture stronger posterior dependencies, while MCMC could be used on a smaller subset to check whether the variational approximation is biased. Finally, the model could be evaluated under a stricter cold-start split, where some users or movies are held out completely. This would test a harder and more realistic recommendation setting than the current held-out-rating split.

Declaration of Generative AI Use

Generative AI tools were used as support for brainstorming the project structure and implementation, debugging code, and improving the wording of the notebook and report. All modelling choices, implementation decisions, results, figures, and conclusions were checked by me, and I take full responsibility for the submitted work.

References

[1] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

[2] J. Winn, C. M. Bishop, T. Diethe, J. Guiver, and Y. Zaykov, *Model-Based Machine Learning*. CRC Press, 2019.

[3] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, pp. 1–19, 2015.

[4] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[5] E. Bingham *et al.*, "Pyro: Deep Universal Probabilistic Programming," *Journal of Machine Learning Research*, vol. 20, no. 28, pp. 1–6, 2019.